pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

Reranking with a Cross-Encoder

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: reranking with a cross-encoder

A pool of 20, rescored

Chunks	1,158 at 512/64
Pool	first 20 of the hybrid list
Top 3 hold key	16 of 23
Top 20 hold key	22 of 23
Model	scoring only; no request sent

Today

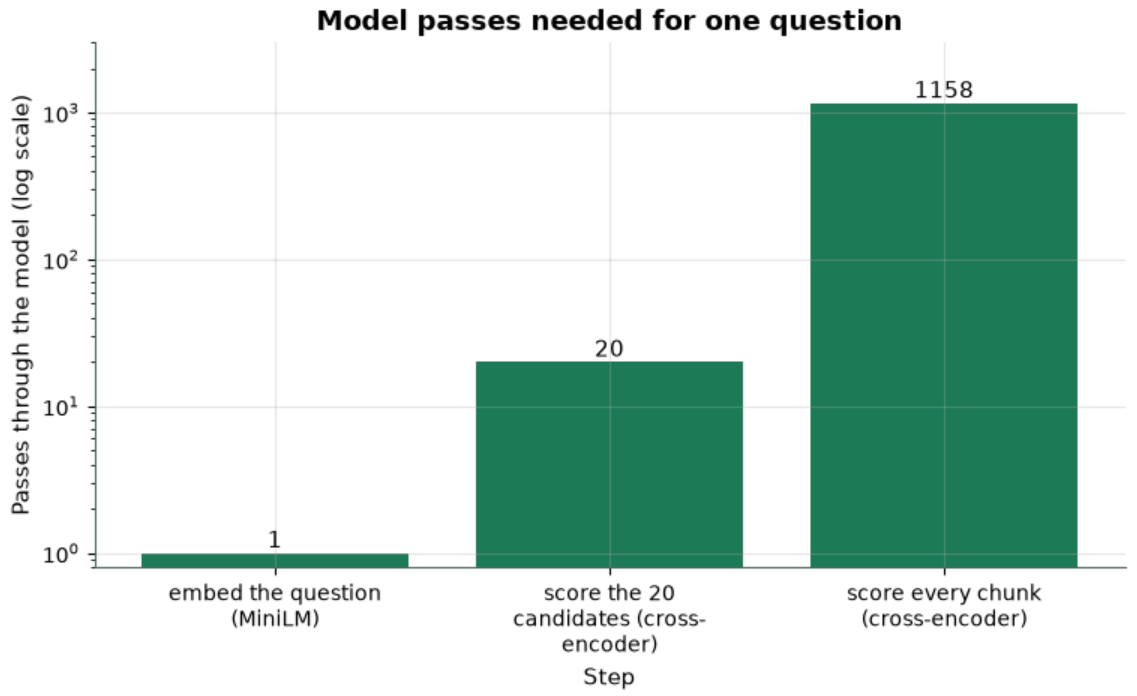
1. Two ways to score a pair
2. The reranker, and its card
3. Rerank 20 to 3; every move as a slope chart
4. candidates and k; two different fixes
5. What it costs; a score is not a probability
6. When reranking does nothing

The pool is a ceiling; name the rule; choose nothing yet.

How a cross-encoder works

The problem	A first search is fast because the chunks' vectors were made ahead of time; the question and the chunk never meet inside the model
The method	The question and one chunk as a single input through a small transformer; one number for the pair; a word in each can be weighed against the other
The price	Nothing can be prepared ahead of time: a fresh pass for every pair, so it runs on a short list, never on all the chunks
You control	Which model, how many candidates it sees, how many chunks it keeps
What comes out	A score per pair, used to reorder the candidates; never a probability

Bi-encoder and cross-encoder



What each takes in

Bi-encoder (MiniLM): the question or the chunk, alone, as 384 numbers

Score: a dot product of two vectors made apart

Cross-encoder: the question and the chunk, together, one input

Score: one number for the pair

A cross-encoder follows a first search; it never replaces it.

cross-encoder/ms-marco-MiniLM-L6-v2

Model card, read 13 October 2026

Licence: apache-2.0

Trained on the MS Marco Passage Ranking task

Training data: non-commercial research purposes only

1,800 documents a second on a V100 GPU (card)

6 layers, 22,713,601 parameters, 512 word pieces

The card's own example (Berlin)

```
scores here [ 8.61 -4.32]
```

```
card prints [ 8.607138 -4.320078]
```

one pair = one input of 34 word pieces

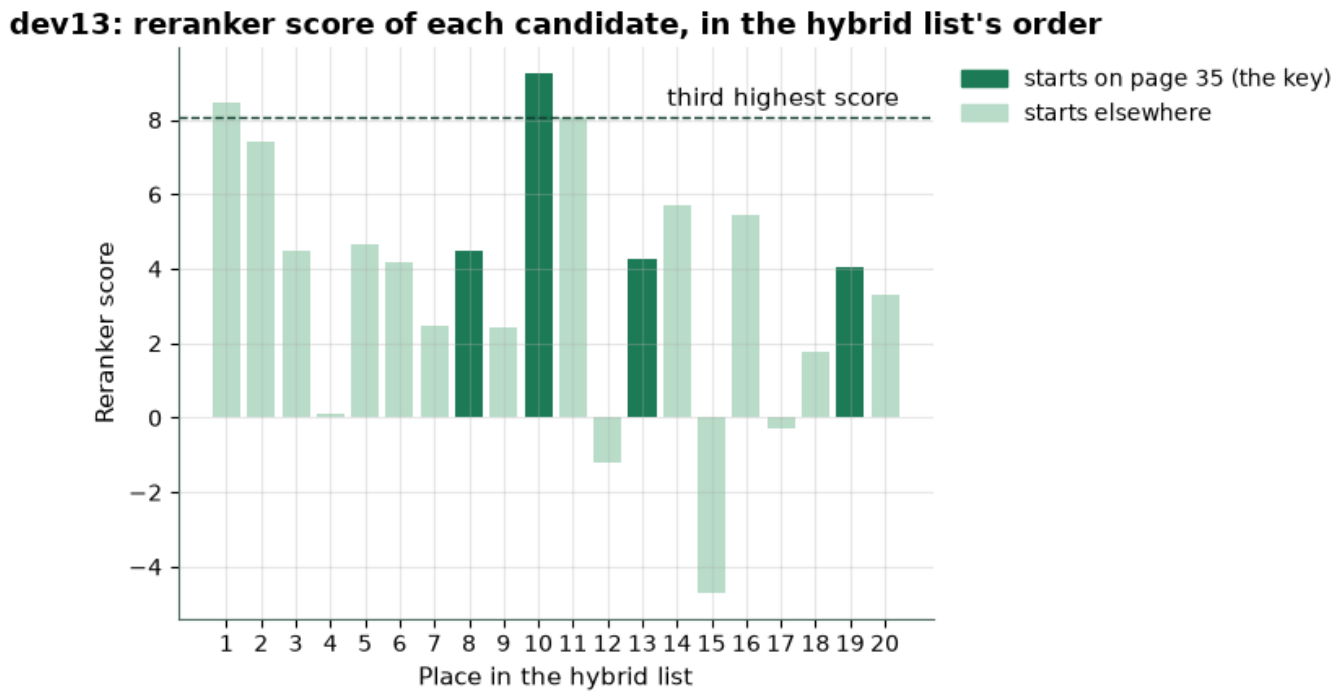
Nogueira and Cho, Passage Re-ranking with BERT, arXiv 1901.04085, submitted 13 January 2019 (arXiv record, read 13 October 2026). The paper itself was not read.

The weights are Apache-2.0; the training data has its own terms. Read both before any use at work.

How reranking works

The problem	The first search puts the right chunk in its first twenty more often than in its first three
The method	Take the first candidates chunks of the hybrid list; score every (question, chunk) pair in one batch; sort by score; the first k go to the model
The ceiling	A chunk that is not among the candidates cannot be rescored into the top k: pool size trades a higher ceiling against more pairs
You control	candidates and k
What comes out	k chunks with the reranker's score in place of the first search's, and the place each had before kept beside it

dev13: cash and cash equivalents

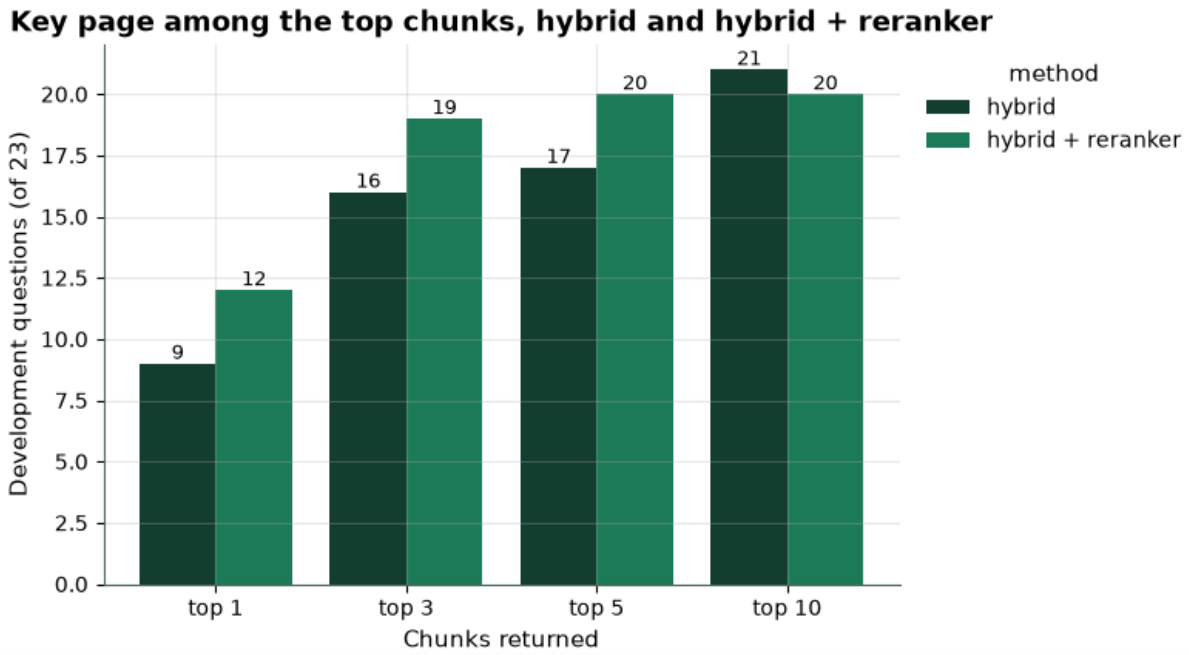


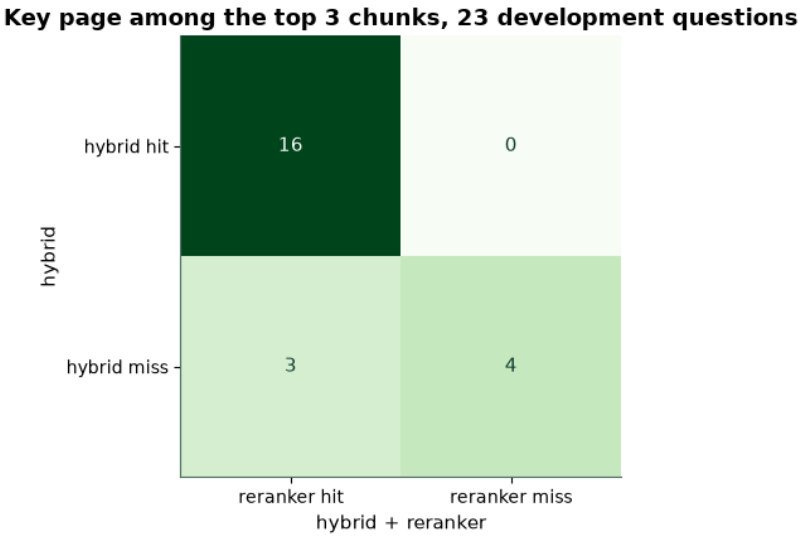
The five largest scores

place	hybrid	page	score
1	10	35	9.24
2	1	59	8.45
3	11	93	8.08
4	2	93	7.40
5	14	59	5.73

The key page chunk: hybrid place 8, reranker place 1, score 9.24. Top three: 9.24, 8.45, 8.08; lowest of the 20: -4.69.

Hits before and after



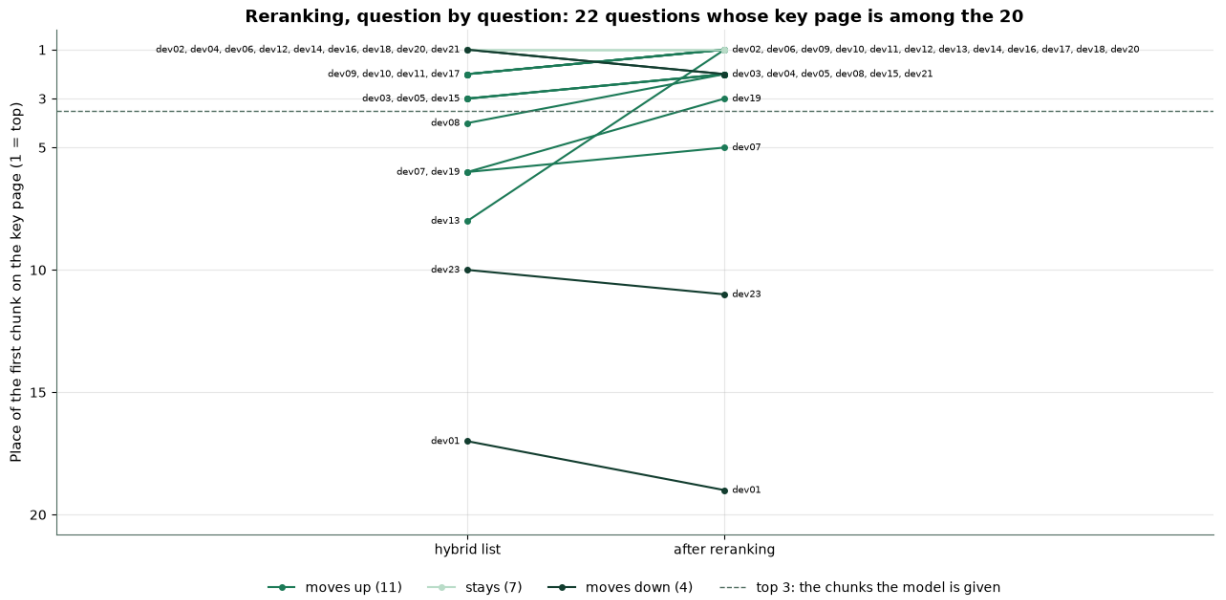


method	top 1	top 3	top 5	top 10
hybrid	9	16	17	21
hybrid + reranker	12	19	20	20

gains dev08, dev13, dev19; loses none

key page moves up 11, stays 7, down 4

Every question's move, as a slope chart



How to read it

Place 1 is at the top; a line that slopes down the page moves up the list

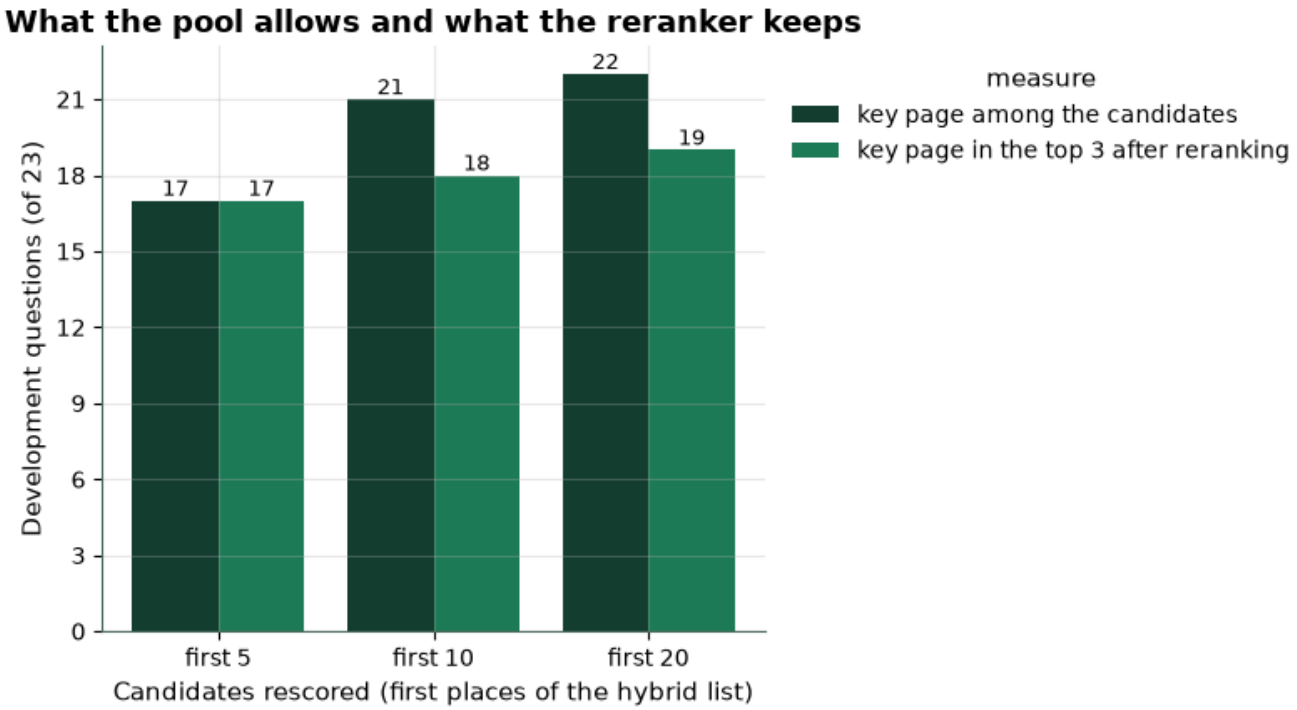
Lines crossing the dashed line into the top 3: 3 (dev08, dev13, dev19); none cross it the other way

Up 11, same 7, down 4: most lines are short

The reranker agrees with the fused order more often than it overturns it

One line per question whose key page is among the 20 candidates: 22 of 23.

The pool is a ceiling



Pool and top 3

first 5

in the pool 17; in the top 3 17

first 10

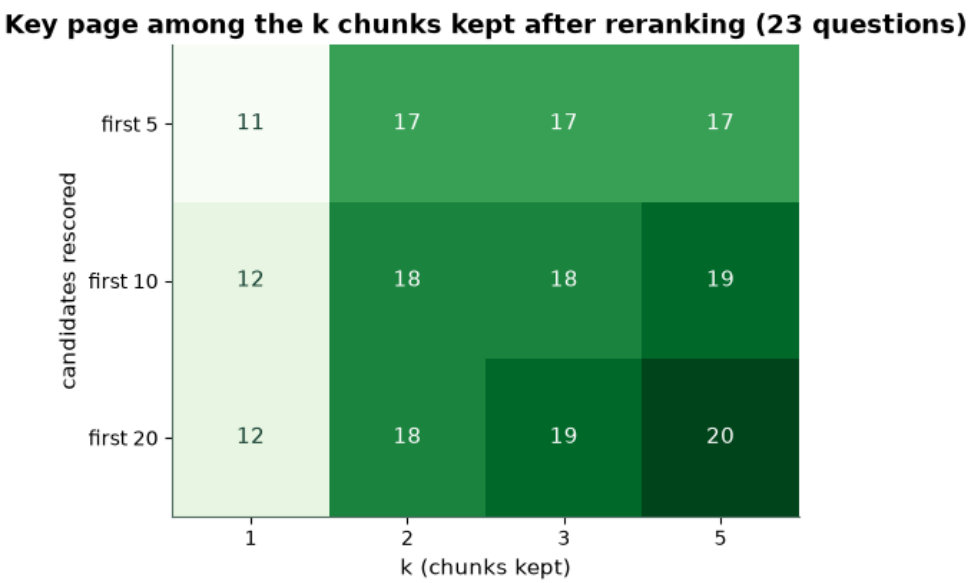
in the pool 21; in the top 3 18

first 20

in the pool 22; in the top 3 19

No reranker can place a page in the top 3 if no chunk of it is a candidate.

candidates and k together, and every setting of the step

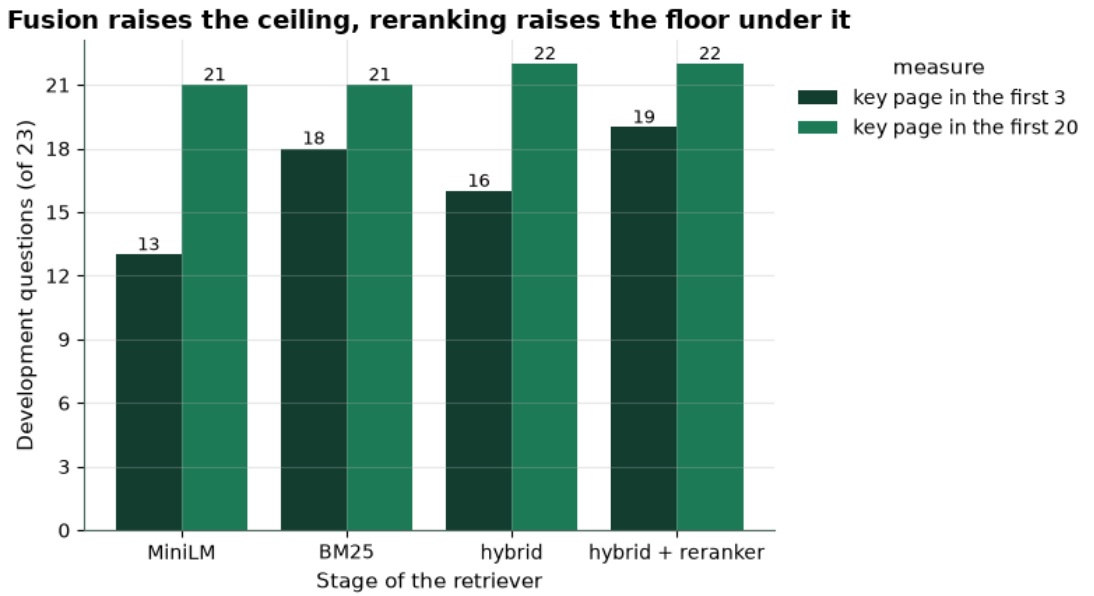


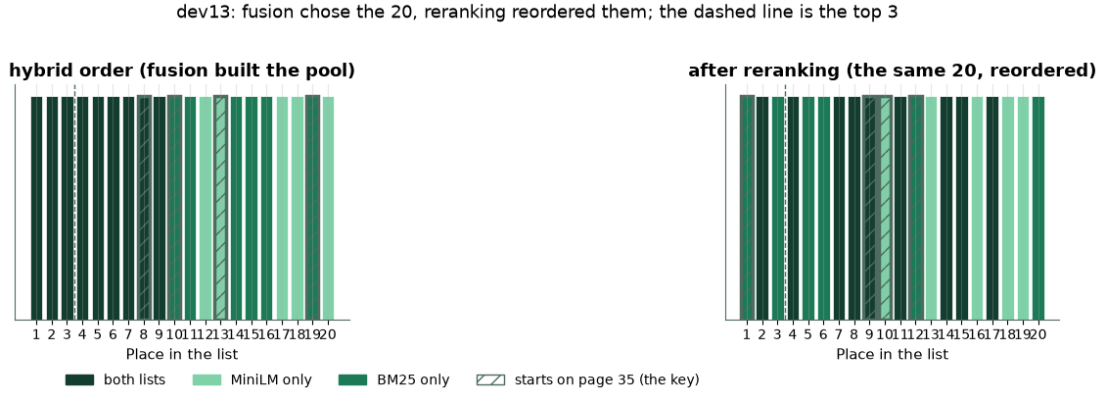
Where	Setting	Default	The course sets
`rerank()`	**`model`**	`required`	load_reranker(), the pinned MS ...
`rerank()`	**`question`**	`required`	each question of the set
`rerank()`	**`candidates`**	`required`	the first 20 of the hybrid list
`rerank()`	**`k`**	`required`	3
`RagSettings`	**`k`**	`3`	3
`RagSettings`	**`candidates`**	`20`	the first 20 of the hybrid list
`RagSettings`	**`rerank`**	`'none'`	none on days 5 and 6; cross-enc...
`CrossEncoder`	**`max_seq_length`**	`512`	the model's own (not changed)

Twelve settings: 11 to 20 of 23; 19 at 20 candidates, k = 3. A larger k helps or holds; a larger pool helps most at k = 5.

Defaults read with inspect.signature and dataclasses.fields; max_seq_length is the model's own limit.

Fusion and reranking are two different fixes

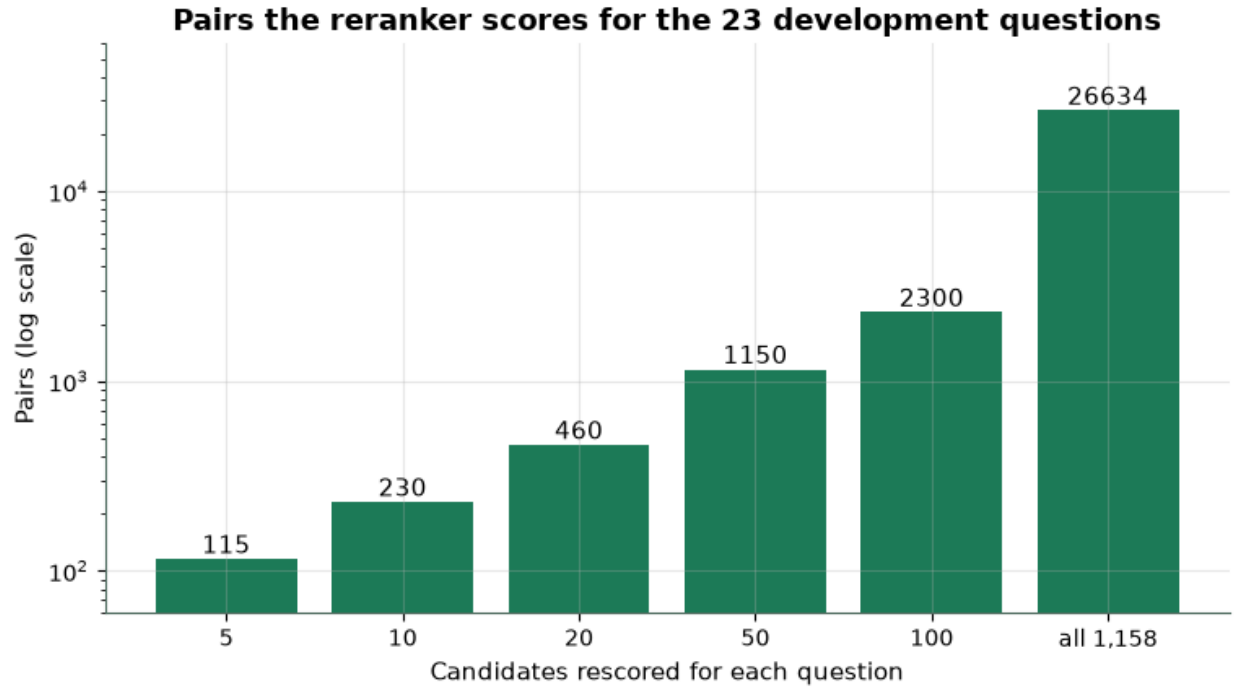




fix	changes	cannot	cost
fusion (day 6)	which chunks are in the pool	put a chunk neither list reaches into the pool	none (places only)
reranking (today)	the order inside the pool	add a chunk to the pool	460 pairs for 23 questions

By stage, first 3 / first 20: MiniLM 13 / 21, BM25 18 / 21, hybrid 16 / 22, hybrid + reranker 19 / 22. Fusion moves the ceiling; reranking moves the floor under it.

The price is pairs



Pairs for 23 questions

5: 115

10: 230

20: 460

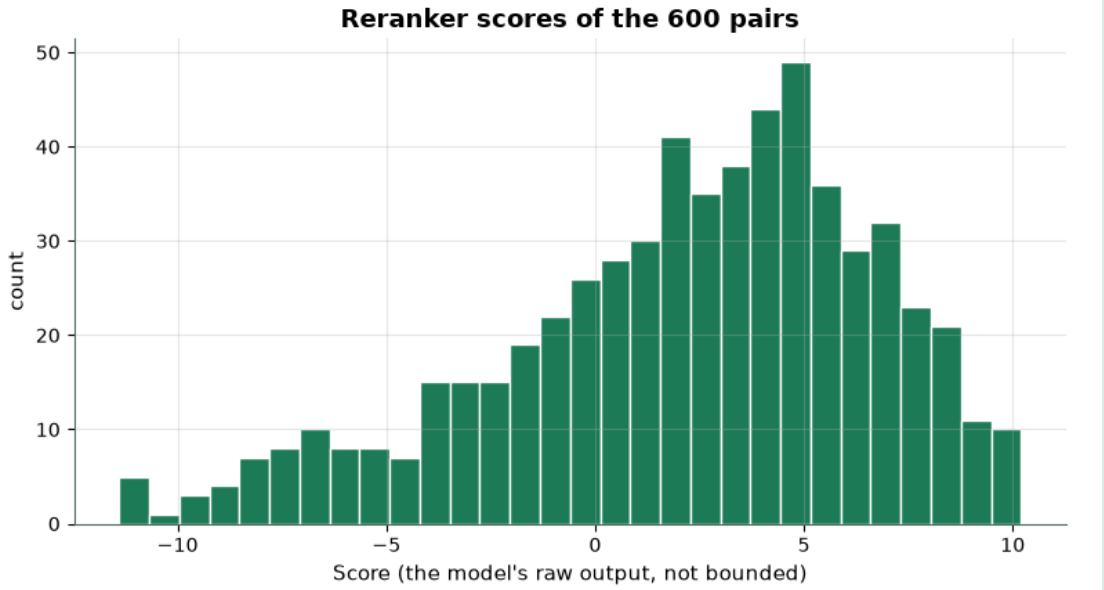
50: 1,150

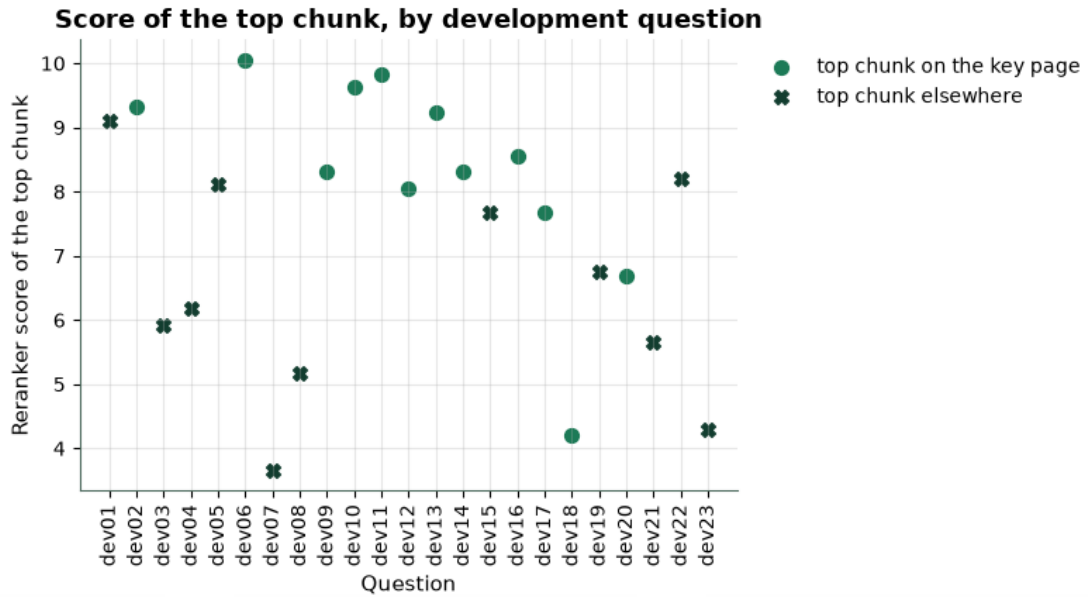
100: 2,300

all 1,158: 26,634

Longest pair: 222 of 512 word pieces. Times vary by computer; pair counts do not.

A score is not a probability





-11.40
lowest

10.20
highest

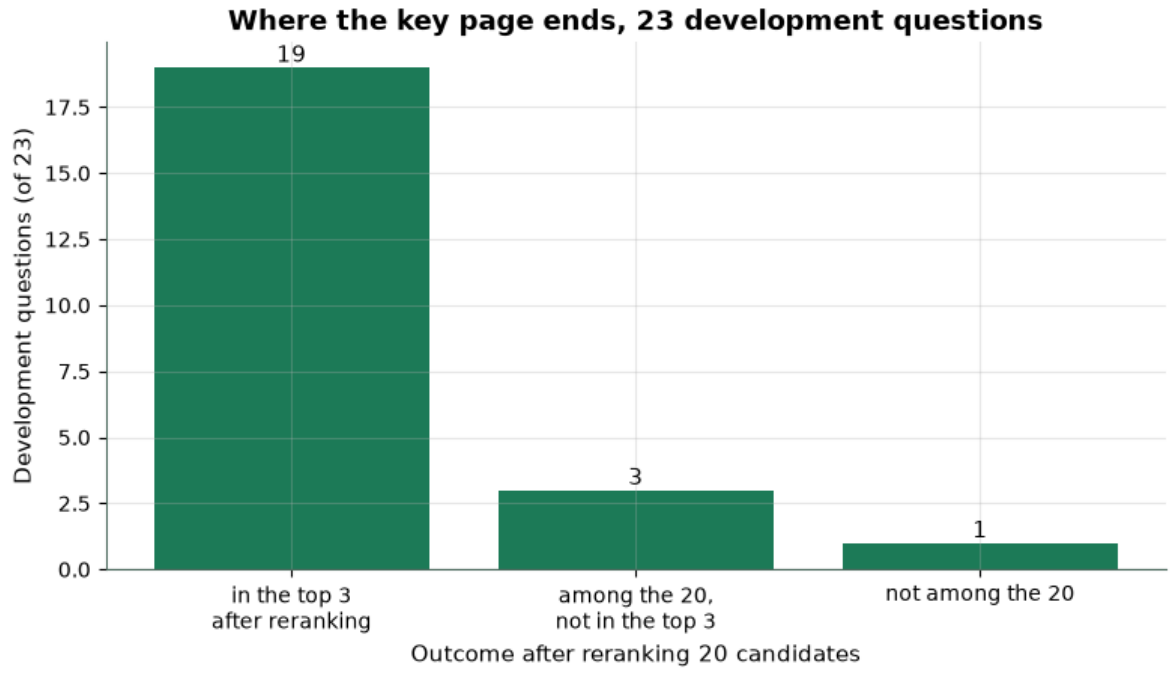
169
of 600 below 0

4.20-10.05
top on key page

3.66-9.11
top elsewhere

The ranges overlap: no cutoff on the top score separates right from wrong.

When reranking does nothing



Place of the first chunk on the key page

id	page	MiniLM	BM25	hybrid	reranker
dev01	37	9	21	17	19
dev07	38	6	3	6	5
dev23	98	62	4	10	11
dev22	52	42	109	-	-

Key page in the top 3: 19; in the 20 but not the top 3: dev01, dev07, dev23; not among the 20: dev22.

Excel side by side, and Git

Excel side by side (tested in Excel)

`=RANK(D2,$D$2:$D$21)`	place after reranking; equal to rerank for 20 chunks
`=SUMPRODUCT(($B$2:$B$21<=3)*($C$2:$C$21=$H$1))>0`	FALSE (key page in the hybrid top 3), equal to Python
`=SUMPRODUCT(($E$2:$E$21<=3)*($C$2:$C$21=$H$1))>0`	TRUE (key page in the top 3 after reranking), equal to Python
`=INDEX($A$2:$A$21,MATCH(G2,$E$2:$E$21,0))`	the chunks in places 1, 2, 3: the same three ids
`=SORTBY($A$2:$A$21,$D$2:$D$21,-1)`	the list sorted by score: the same order

Git: rag_results.csv with 2 rows, then a third run and its diff; 40 tests.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the candidates are sorted from the smallest score, so the function keeps the three lowest.

Code that runs is not the same as code that is right.

What you can do now

- A cross-encoder scores a pair as one input
- Load a model at a pinned revision; quote its terms and dates
- Rerank 20 to 3; name the rule of every count; draw the moves as a slope chart
- The pool is a ceiling; sweep candidates and k together; pairs are the price
- Fusion changes the pool, reranking its order; a score is not a probability

Week 2, Day 3 Tuning the pipeline: a results grid